

DSS5201 Data Visualisation

Week 7 Relational Data

David CHEW

Semester 2 AY25/26

Recap: Tidy Data

Data rarely arrive in the condition that we need them.

They need to be reshaped and reformatted.

“Tidy” Table

Business Unit	Year	Quarter	Budget
Sales	2000	Q1	2,500,000
Marketing	2000	Q1	1,000,000
Sales	2000	Q2	2,750,000
Marketing	2000	Q2	1,250,000
Sales	2000	Q3	3,000,000
Marketing	2000	Q3	4,000,000
Sales	2000	Q4	2,000,000
Marketing	2000	Q4	500,000
Sales	2001	Q1	2,500,000
Marketing	2001	Q1	1,500,000

“UnTidy” Table

Past and projected budgets for WidgetCo.'s Sales and Marketing Org.						
Contact JDoe@widgets.ca for more information.						
Business Unit	Year					
	2000				2001	
	Q1	Q2	Q3	Q4	Q1	Q2*
Sales	2,500,000	2,750,000	3,000,000	2,000,000	2,500,000	3,000,000
Marketing	1,000,000	1,250,000	4,000,000	500,000	1,500,000	1,750,000
double check this number - JD						
*Projected Numbers						

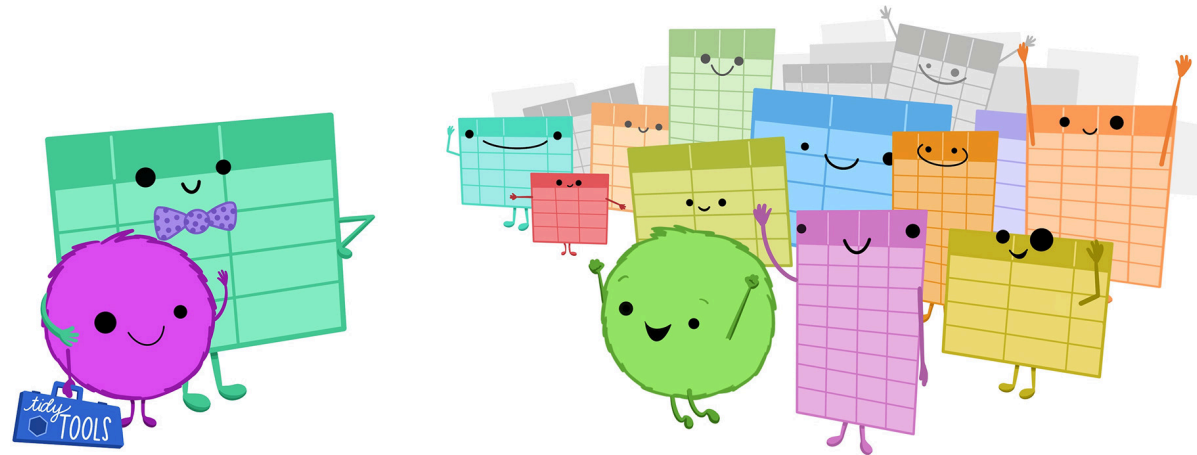
The tidy table is ready for use. The untidy table is not.

This Week: Relational Data

- Relational Data
- Relations
- Joins
- Row Filtering
- Concatenation

Relational Data

When One Table is Not Enough



In practice, data rarely live in a single table.

- Typically, information is spread across multiple files that need to be combined to answer the questions we are interested in.
- A collection of related tables is known as **relational data**.

When One Table is Not Enough

restaurant				health inspections					rating		
name	id	address	type	name	id	inspection_date	inspector	score	name	id	stars
Taco Stand	AH13JK	1 Main St.	Mexican	Taco Stand	AH13JK	2018-08-21	Sheila	97	Taco Stand	AH13JK	4.9
Pho Place	JJ29JJ	192 Street Rd.	Vietnamese	Pho Place	JJ29JJ	2018-03-12	D'eonte	98	Pho Place	JJ29JJ	4.8
Taco Stand	XJ11AS	18 W. East St.	Fusion	Pho Place	JJ29JJ	2018-01-02	Monica	66	Taco Stand	XJ11AS	4.2
Pizza Heaven	CI21AA	711 K Ave.	Italian	Taco Stand	XJ11AS	2018-12-16	Mark	43	Pizza Heaven	CI21AA	4.7
				Pizza Heaven	CI21AA	2018-08-21	Anh	99			

Across multiple data files, we have information on

- Location and type of cuisine.
- Health and safety inspections results.
- Online ratings on the restaurant.

The tables share a common column (**id**) that links them together.

The Advantages of Relational Data

Storing data across multiple files has several benefits:

- **Efficient Data Storage**
 - Avoid repeating the same information multiple times.
 - As data sets grow, splitting them in multiple linked tables keep the data manageable.
- **Easier Data Updates**
 - If we need to update information, we can make the change in a single file.
- **Privacy**
 - Different users access only the tables they need.

The nycflights13 Data Set

Today, we work with five related tables about flights from New York City in 2013.

1. `flights`: All flights that departed NYC (i.e. JFK, LGA or EWR) in 2013.
2. `airlines`: Look up airline names from their carrier codes.
3. `airports`: Useful metadata about airports.
4. `planes`: Plane metadata for all tailnumbers found in the FAA aircraft registry.
5. `weather`: Hourly meteorological data for LGA, JFK and EWR.

The nycflights13 Data Set

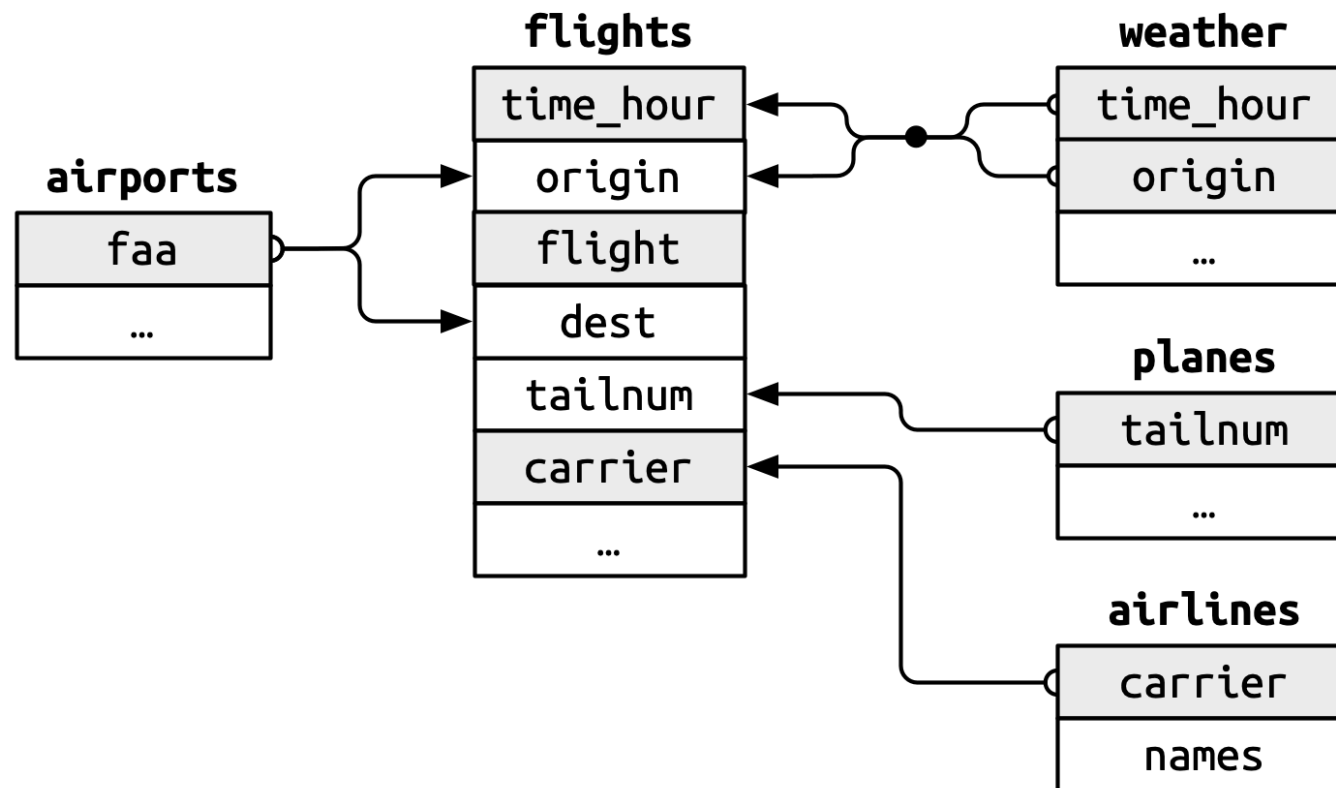
Download the files from Canvas and place them in your `data/nycflights13` directory:

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 flights = pd.read_csv("../data/nycflights13/flights.csv.zip")
5 airlines = pd.read_csv("../data/nycflights13/airlines.csv")
6 airports = pd.read_csv("../data/nycflights13/airports.csv")
7 planes = pd.read_csv("../data/nycflights13/planes.csv")
8 weather = pd.read_csv("../data/nycflights13/weather.csv")
```

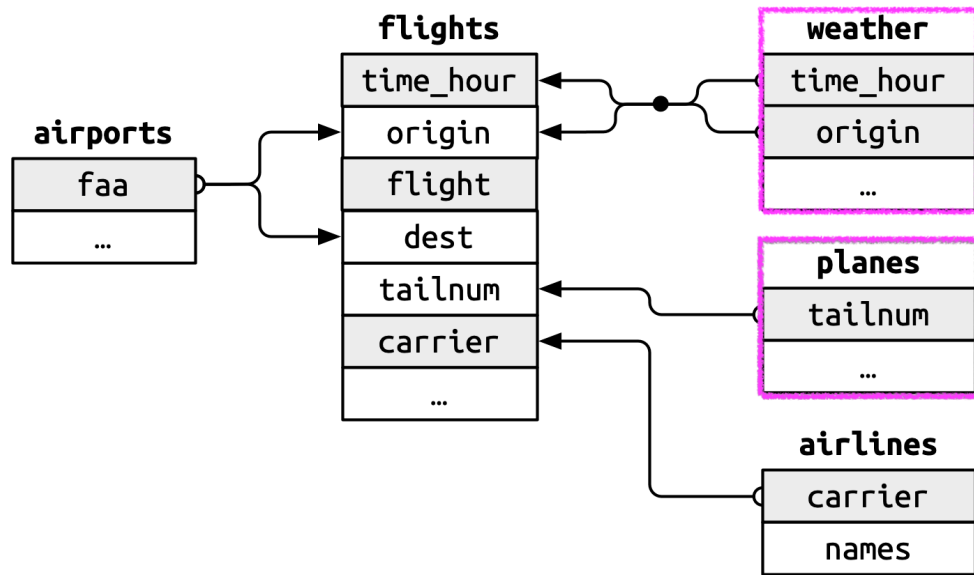


The nycflights13 Data Set

A database schema that identifies the connections between the tables:



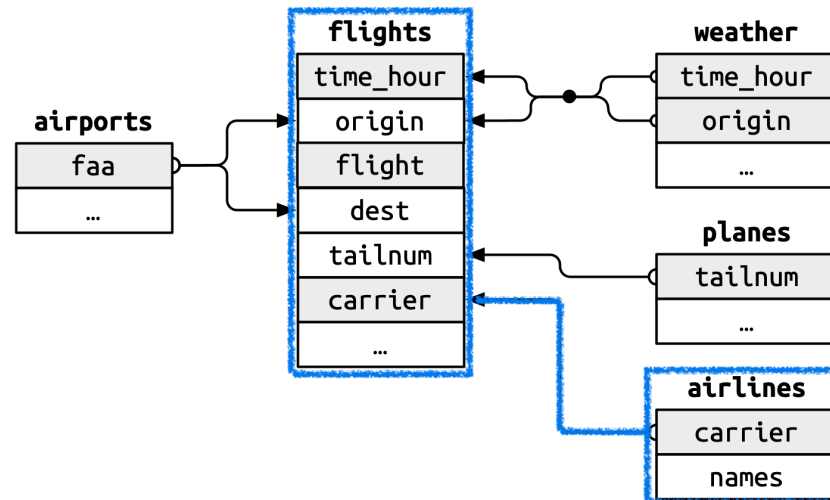
The nycflights13 Data Set: Keys



Keys connect tables together.

- For `weather`, the unique identifier is `time_hour` and `origin`.
- For `planes`, the unique identifier is `tailnum`.

The nycflights13 Data Set: Joins



Each **join** involves a pair of keys:

- The **primary key** uniquely identifies an observation within a table.
 - `airlines['carrier']` is the primary key for `airlines`.
- The **foreign key** links the primary key to another table.
 - `flights['carrier']` is a foreign key linked to `airlines['carrier']`.

Checking Primary Keys: Uniqueness

Once we identify the primary key(s), it is important to verify if

1. It can **uniquely identify** each observation.
2. It has **no missing values**.

How many duplicated `carrier` codes are there in `airlines`?

- Use `.duplicated()` to check for duplication and `.sum()` to count them.

```
1 airlines['carrier'].duplicated().sum()
```

```
np.int64(0)
```

Checking Primary Keys: Uniqueness

From the schema, `time_hour` and `origin` are the primary keys for `weather`.

- If we check this with `.duplicated()`, we see there are 0 duplicates.

```
1 weather.duplicated(subset = ['time_hour', 'origin']).sum()
```

```
np.int64(0)
```

- In contrast, `origin` alone cannot uniquely identify weather observations.

```
1 weather.duplicated(subset = ['origin']).sum()
```

```
np.int64(26112)
```

Checking Primary Keys: Missing Values

We should also check for missing values in primary keys.

- If a value is missing, that row cannot be uniquely identified.
- Use `.isna()` to check for missing values and `.sum()` to count them.

```
1 airlines['carrier'].isna().sum()
```

```
np.int64(0)
```

```
1 weather[['time_hour', 'origin']].isna().sum()
```

```
time_hour    0
```

```
origin       0
```

```
dtype: int64
```

Relations

Relations: One-to-One

x		y	
key	var_x	key	var_y
1	x1	1	y1
2	x2	2	y2
3	x3	4	y3

A **relation** is formed when a primary key in one table corresponds to a foreign key in another.

- Ideally, relations are **one-to-one**.
 - Each key in Table x matches at most one key in Table y.
- Example: Each student has a matriculation number; and each matriculation number identifies one student.

Relations: One-to-Many or Many-to-One

x		y	
key	var_x	key	var_y
1	x1	1	y1
2	x2	2	y2
3	x3	2	y3

x		y	
key	var_x	key	var_y
1	x1	1	y1
2	x2	2	y2
2	x3	4	y3

In real-world data sets, relations are often **one-to-many** or **many-to-one**.

- Each key in Table x corresponds to multiple rows in y, or vice versa.
- Example: Each flight is linked to one airline, but each airline has many flights.

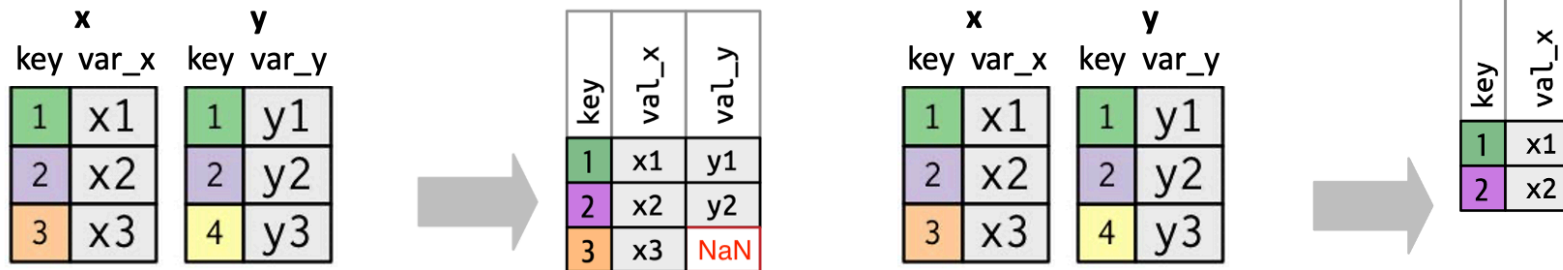
Relations: Many-to-Many

x		y	
key	var_x	key	var_y
1	x1	1	y1
2	x2	2	y2
2	x3	2	y3

Relations can sometimes be **many-to-many**.

- Example: Each airline flies to many airports, and each airport serves many airlines.
- If we join such tables directly, we create a **Cartesian product**. In most cases, this will be problematic. 
 - To avoid this issue, ensure that the key is unique in at least one table.

Relations Between Tables



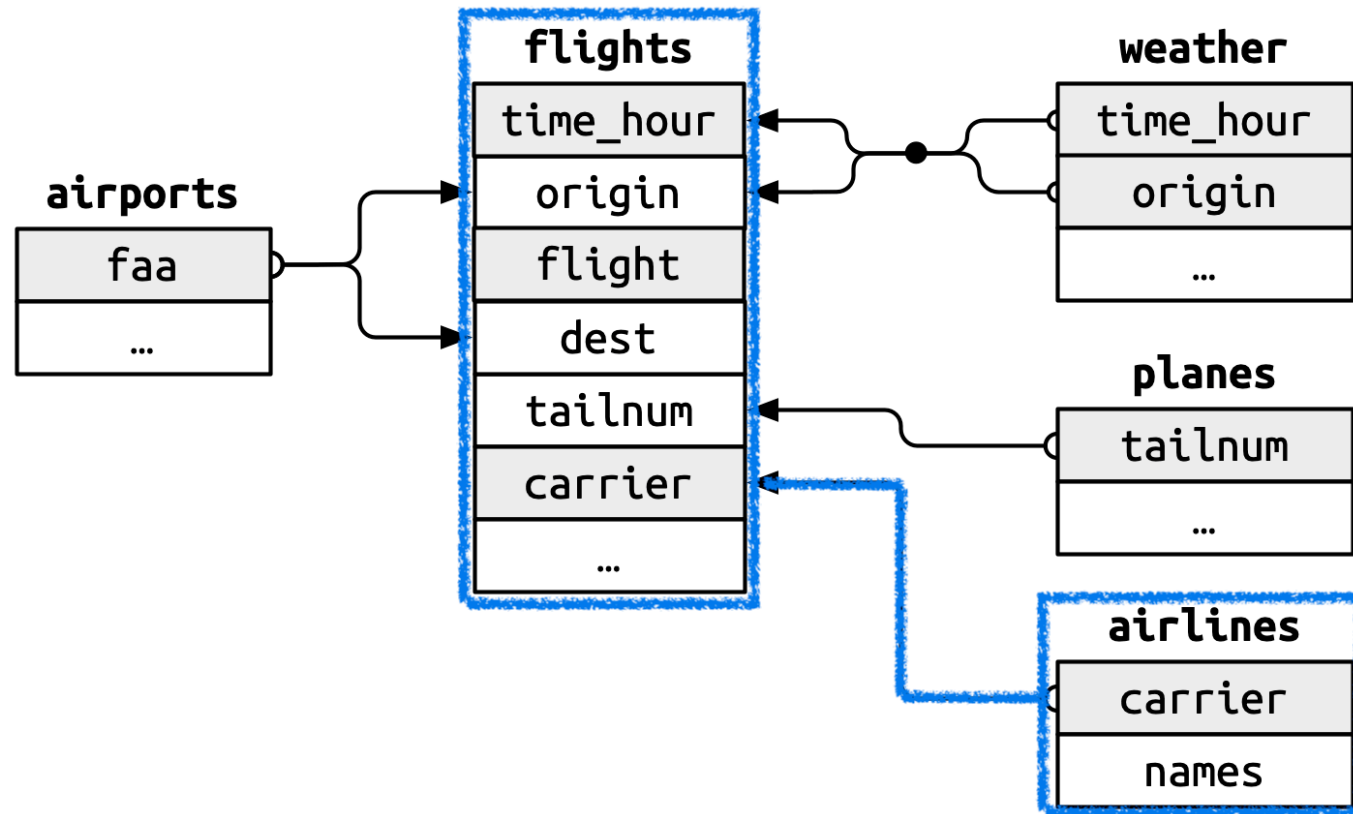
When working with relational data, we often merge or filter tables:

- **Joins/Merges** : Match rows from two tables using keys to add new columns.
 - Types of join: *inner*, *left*, *right*, *outer*
- **Row Filtering**: Filter one table based on matches in another.
- **Concatenation**: Bind rows or columns from multiple tables.

Joins/Merges

Joins/Merges

Let's continue to work with the five tables from the `nycflights13` data set. We will first join the tables `flights` and `airlines` via the key `carrier`.



Joins/Merges

For ease of discussion, we consider `flights` with a subset of its columns.

```
1 flights2 = flights[['time_hour', 'origin', 'dest', 'tailnum', 'carrier']]
2 flights2.head()
```

	time_hour	origin	dest	tailnum	carrier
0	2013-01-01T10:00:00Z	EWR	IAH	N14228	UA
1	2013-01-01T10:00:00Z	LGA	IAH	N24211	UA
2	2013-01-01T10:00:00Z	JFK	MIA	N619AA	AA
3	2013-01-01T10:00:00Z	JFK	BQN	N804JB	B6
4	2013-01-01T11:00:00Z	LGA	ATL	N668DN	DL

.merge() by keys

Let us examine the `airlines` table.

- The primary key in `airlines` is `carrier`.
- We have checked previously that
 - the key is unique, and
 - there are no missing values with this key variable.

```
1 airlines.head()
```

	carrier	name
0	9E	Endeavor Air Inc.
1	AA	American Airlines Inc.
2	AS	Alaska Airlines Inc.
3	B6	JetBlue Airways
4	DL	Delta Air Lines Inc.

.merge() by keys

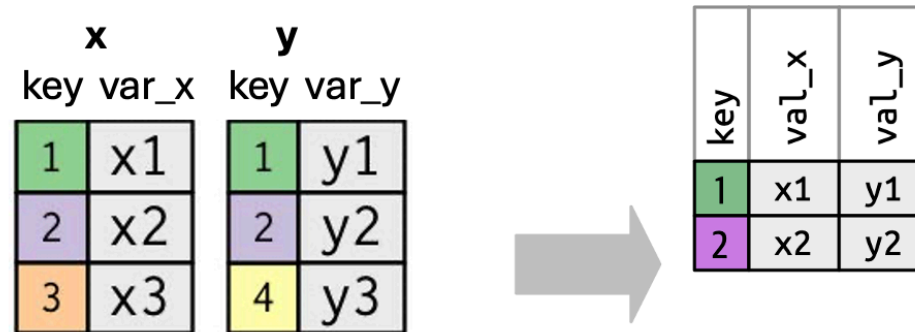
By default, `.merge()` applies an **inner join**.

- `how` specifies the type of join;
- `on` specifies the column names to join on — must exist in both data frames.

```
1 df = flights2.merge(airlines, how = 'inner', on = 'carrier')
2 df.head()
```

	time_hour	origin	dest	tailnum	carrier	name
0	2013-01-01T10:00:00Z	EWR	IAH	N14228	UA	United Air Lines Inc.
1	2013-01-01T10:00:00Z	LGA	IAH	N24211	UA	United Air Lines Inc.
2	2013-01-01T10:00:00Z	JFK	MIA	N619AA	AA	American Airlines Inc.
3	2013-01-01T10:00:00Z	JFK	BQN	N804JB	B6	JetBlue Airways
4	2013-01-01T11:00:00Z	LGA	ATL	N668DN	DL	Delta Air Lines Inc.

Inner Join



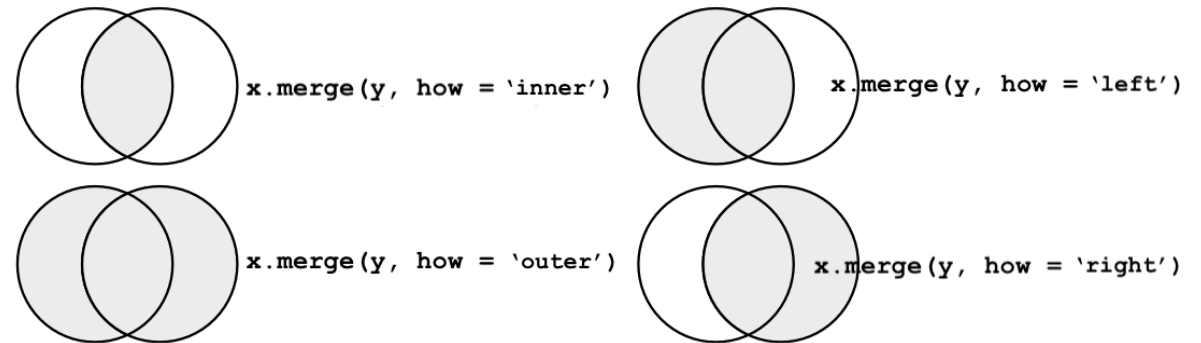
An inner join (`how = 'inner'`) matches observations whenever keys are equal.

- Observations that appear in **both** tables are kept and unmatched ones are removed.
 - Rows with keys 1 and 2 are kept.
 - Rows with keys 3 (in Table x) and 4 (in Table y) are removed.
- Columns from right table will be added to the resulting data frame.
 - The column `var_y` is added.

Types of Join

There are **4 types of joins**:

- Inner join
- Left outer join
- Right outer join
- Full outer join



To understand how each of them works, we will simplify our working data and use visual representations.

```
1 x = flights[['origin', 'dest', 'tailnum', 'carrier']].head(4)
2 x
```

	origin	dest	tailnum	carrier
0	EWR	IAH	N14228	UA
1	LGA	IAH	N24211	UA
2	JFK	MIA	N619AA	AA
3	JFK	BQN	N804JB	B6

```
1 y = airlines[airlines['carrier'].isin(['UA', 'AA', '9E', 'WN'])]
2 y
```

	carrier	name
0	9E	Endeavor Air Inc.
1	AA	American Airlines Inc.
11	UA	United Air Lines Inc.
14	WN	Southwest Airlines Co.

Understanding Joins

The tables we just created look like:

Table x				Table y	
origin	dest	tailnum	carrier	carrier	names
			A	A	
			B	B	
			C	D	
				E	
				...	

- The colored column represents the **key** variable: `carrier`.
- In this example, we have a single key variable, but the idea generalizes to multiple keys.

Defining a Join

A join connects each row in Table x to 0, 1 or more rows in Table y via a key.

- Number of links = Number of matches.

Table x					Table y	
origin	dest	tailnum	carrier		carrier	names
			A	to	A	
			B	to	B	
			C		D	
					E	
					...	

inner join

origin	dest	tailnum	carrier	names
			A	
			B	

- An inner join keeps *carriers* that appear in both tables: A and B.
- Different types of joins can lead to different results.

Inner Join

Let us start with the inner join (`how = 'inner'`).

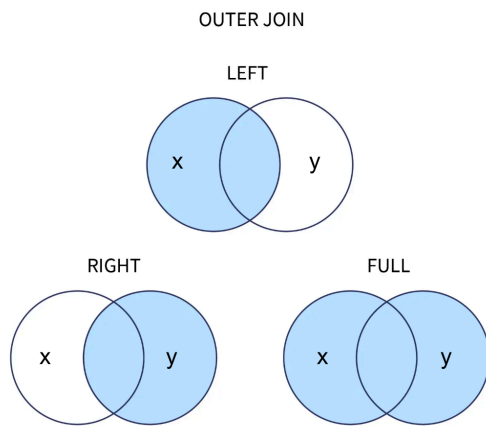
- An **inner join** keeps only rows where the key appears in both tables.
 - Carriers `UA` and `AA` are kept.
- All unmatched rows are removed.
 - Any `carrier` that appears only in one table are removed.
- Additional columns from the right table will be added to the result.
 - The `name` column is added.

```
1 x.merge(y, how = 'inner', on = 'carrier')
```

	origin	dest	tailnum	carrier	name
0	EWR	IAH	N14228	UA	United Air Lines Inc.
1	LGA	IAH	N24211	UA	United Air Lines Inc.
2	JFK	MIA	N619AA	AA	American Airlines Inc.

Three Types of Outer Joins

An **outer join** keeps observations that appear in **at least one** of the tables.



- **Left outer join:** Keeps all rows in x, including those not matched in y.
- **Right outer join:** Keeps all rows in y, including those not matched in x.
- **Full outer join:** Keeps all rows in both tables, regardless of matches.

Left Outer Join

```
1 x.merge(y, how = 'left', on = 'carrier')
```

	origin	dest	tailnum	carrier	name
0	EWR	IAH	N14228	UA	United Air Lines Inc.
1	LGA	IAH	N24211	UA	United Air Lines Inc.
2	JFK	MIA	N619AA	AA	American Airlines Inc.
3	JFK	BQN	N804JB	B6	NaN

- A left join keeps all rows in x, the table on the left.
- If a key is not found in y, the missing values are filled with NaNs.

Right Outer Join

```
1 x.merge(y, how = 'right', on = 'carrier')
```

	origin	dest	tailnum	carrier	name
0	NaN	NaN	NaN	9E	Endeavor Air Inc.
1	JFK	MIA	N619AA	AA	American Airlines Inc.
2	EWR	IAH	N14228	UA	United Air Lines Inc.
3	LGA	IAH	N24211	UA	United Air Lines Inc.
4	NaN	NaN	NaN	WN	Southwest Airlines Co.

- A right join keeps all rows from y, the table on the right.
- If a key is not found in x, the missing values are filled with NaNs.

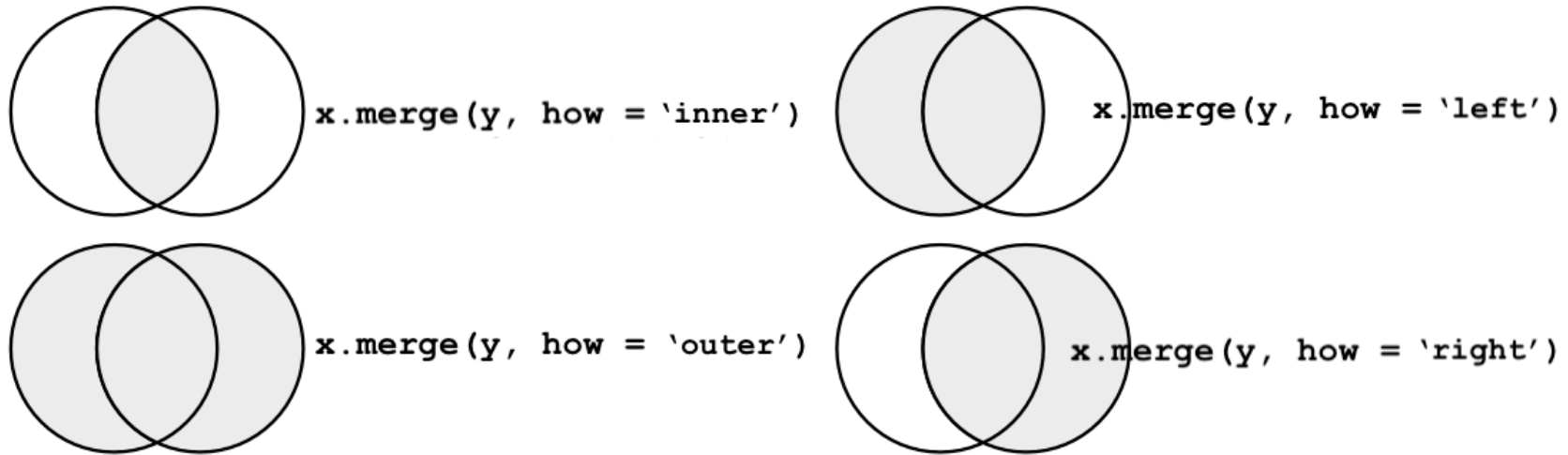
Full Outer Join

```
1 x.merge(y, how = 'outer', on = 'carrier')
```

	origin	dest	tailnum	carrier	name
0	NaN	NaN	NaN	9E	Endeavor Air Inc.
1	JFK	MIA	N619AA	AA	American Airlines Inc.
2	JFK	BQN	N804JB	B6	NaN
3	EWR	IAH	N14228	UA	United Air Lines Inc.
4	LGA	IAH	N24211	UA	United Air Lines Inc.
5	NaN	NaN	NaN	WN	Southwest Airlines Co.

- A full join returns all rows from both x and y.
- If a key is missing in either table, unmatched entries are filled with NaNs.

Left Outer Join: The Default join



The most common join is the **left outer join**.

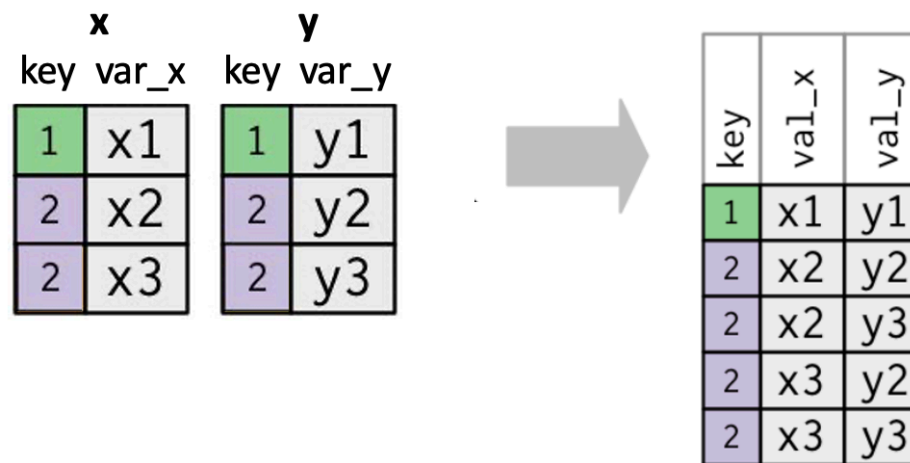
- Put the main table x on the left.
- Ensures that all rows from x are preserved even if there is no match in y.

Always use the left outer join as your default join.

Joints: Avoid Duplicated Keys

When joining tables, we should avoid duplication in the key columns.

- If both table contain duplicated keys, the join produces all possible combinations: the Cartesian product.



- 🚨 These are problematic as they can result in row explosion after the join.
- Can be a source of bugs and will have a large impact on code performance.
 - In most cases, ensure to have unique keys for at least one of your tables.

Joints: Specifying Join Keys

Let look at the `flights2` data once more.

```
1 flights2.head()
```

	time_hour	origin	dest	tailnum	carrier
0	2013-01-01T10:00:00Z	EWR	IAH	N14228	UA
1	2013-01-01T10:00:00Z	LGA	IAH	N24211	UA
2	2013-01-01T10:00:00Z	JFK	MIA	N619AA	AA
3	2013-01-01T10:00:00Z	JFK	BQN	N804JB	B6
4	2013-01-01T11:00:00Z	LGA	ATL	N668DN	DL

Joints: Specifying Join Keys

There are several ways to specify the join keys.

1. The best practice is always to explicitly specify `on = 'key'`.

```
1 flights2.merge(airlines, how = 'left', on = 'carrier').head()
```

	time_hour	origin	dest	tailnum	carrier	name
0	2013-01-01T10:00:00Z	EWR	IAH	N14228	UA	United Air Lines Inc.
1	2013-01-01T10:00:00Z	LGA	IAH	N24211	UA	United Air Lines Inc.
2	2013-01-01T10:00:00Z	JFK	MIA	N619AA	AA	American Airlines Inc.
3	2013-01-01T10:00:00Z	JFK	BQN	N804JB	B6	JetBlue Airways
4	2013-01-01T11:00:00Z	LGA	ATL	N668DN	DL	Delta Air Lines Inc.

Joints: Specifying Join Keys

2. When the keys are not explicitly defined, `.merge()` uses all common variables across **both** tables as join keys.

This can be problematic: 

- Keys may not have the same names in both tables.
- ... even if they do, columns with the same names may not actually be keys.

The following code will produce a `MergeError`.

```
1 flights2.merge(airports, how = 'left') # MergeError
```



Joins: Specifying Join Keys

3. When the key column names differ between tables, we must explicitly specify:

- `left_on`: The key column on the left table.
- `right_on`: The key column on the right table.

The following joins the `origin` column from `flights2` to `faa` from `airports`.

```
1 flights2.merge(airports, how = 'left', left_on = 'origin', right_on = 'faa').head
```

	time_hour	origin	dest	tailnum	carrier	faa	name	lat	lon	alt	tz	dst	
0	2013-01-01T10:00:00Z	EWR	IAH	N14228	UA	EWR	Newark Liberty Intl	40.692500	-74.168667	18	-5	A	Ame
1	2013-01-01T10:00:00Z	LGA	IAH	N24211	UA	LGA	La Guardia	40.777245	-73.872608	22	-5	A	Ame
2	2013-01-01T10:00:00Z	JFK	MIA	N619AA	AA	JFK	John F Kennedy Intl	40.639751	-73.778925	13	-5	A	Ame

Row Filtering

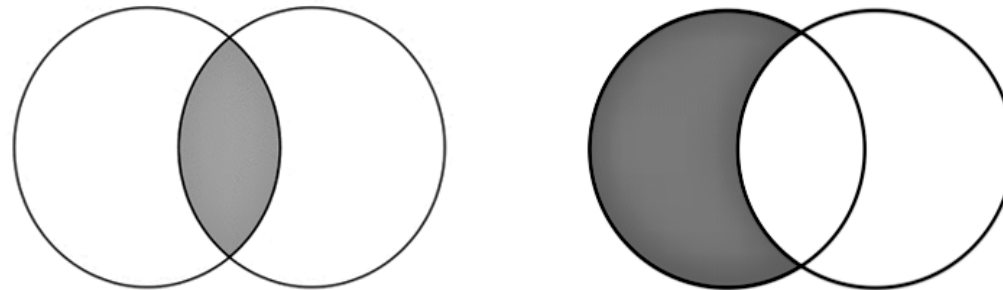
Row Filtering

Row filtering match observations like joins.

- But instead of combining tables, it selects/removes rows from the left table.

Two common uses:

- `.isin()`: Keep only rows in x where the key exists in y.
- Negation of `.isin()`: Remove rows in x where the key exists in y.



.isin()

`.isin()` keeps only the rows in x that **matches** to y based on the key.

- Unlike `.merge()`, it does not add new columns in the resulting table.

Table x				Table y	
origin	dest	tailnum	carrier	carrier	names
			A	A	
			B	B	
			C	D	
				E	
				...	

isin()

origin	dest	tailnum	carrier
			A
			B

.isin()

Flights to Top 3 Destinations

Suppose we want to subset `flights2` to include only flights to the most popular destinations.

1. Identify the top three destinations.

```
1 top_dest = (flights2['dest']  
2             .value_counts()  
3             .nlargest(3)  
4             .reset_index(name = 'count'))  
5 top_dest
```

	dest	count
0	ORD	17283
1	ATL	17215
2	LAX	16174

.isin()

Flights to Top 3 Destinations

2. Use `.isin()` to find all flights whose destination are in the top three list.

```
1 flights3 = flights2[flights2['dest'].isin(top_dest['dest'])]  
2 flights3.head()
```

	time_hour	origin	dest	tailnum	carrier
4	2013-01-01T11:00:00Z	LGA	ATL	N668DN	DL
5	2013-01-01T10:00:00Z	EWR	ORD	N39463	UA
9	2013-01-01T11:00:00Z	LGA	ORD	N3ALAA	AA
12	2013-01-01T11:00:00Z	JFK	LAX	N29129	UA
18	2013-01-01T11:00:00Z	LGA	ATL	N542MQ	MQ

Row Filtering vs Joins

We can compare the results of `.isin()` with a left join:

- `.isin()` filters row only.
- `.merge()` adds extra columns (`count`) from the other table.

```
1 flights4 = flights2.merge(top_dest, how = 'left', on = 'dest')
2 flights4.head(6)
```

	time_hour	origin	dest	tailnum	carrier	count
0	2013-01-01T10:00:00Z	EWR	IAH	N14228	UA	NaN
1	2013-01-01T10:00:00Z	LGA	IAH	N24211	UA	NaN
2	2013-01-01T10:00:00Z	JFK	MIA	N619AA	AA	NaN
3	2013-01-01T10:00:00Z	JFK	BQN	N804JB	B6	NaN
4	2013-01-01T11:00:00Z	LGA	ATL	N668DN	DL	17215.0
5	2013-01-01T10:00:00Z	EWR	ORD	N39463	UA	17283.0

Negation of .isin()

This keeps only rows in x that is **not matched** to y.

Table x					Table y	
origin	dest	tailnum	carrier		carrier	names
			A	↗	A	
			B	↘	B	
			C		D	
					E	
					...	

~ isin()

origin	dest	tailnum	carrier
			C

Negation of .isin()

Flights Not to Top 3 Destinations

Flights that did not go to the top three destinations.

```
1 flights5 = flights2[~flights2["dest"].isin(top_dest["dest"])]  
2 flights5.head()
```

	time_hour	origin	dest	tailnum	carrier
0	2013-01-01T10:00:00Z	EWR	IAH	N14228	UA
1	2013-01-01T10:00:00Z	LGA	IAH	N24211	UA
2	2013-01-01T10:00:00Z	JFK	MIA	N619AA	AA
3	2013-01-01T10:00:00Z	JFK	BQN	N804JB	B6
6	2013-01-01T11:00:00Z	EWR	FLL	N516JB	B6

Concatenation

Concatenation: By Rows or By Columns

We can concatenate two data frames with `.concat()`.

- `axis = 0` indicates concatenation by rows.
- `axis = 1` indicates concatenation by columns.

Concatenation: By Rows or By Columns

Consider the following data frames `df1`, `df2` and `df3`:

```
1 df1 = pd.DataFrame(np.arange(6).reshape(3, 2),  
2                     columns=["satu", "dua"])  
3 df2 = pd.DataFrame(6 + np.arange(6).reshape(3, 2),  
4                     columns=["satu", "dua"])  
5 df3 = pd.DataFrame(6 + np.arange(6).reshape(3, 2),  
6                     columns=["tiga", "empat"])  
7  
8 print(df1)  
9 print(df3)  # df3 is the same as df2, only the columns names are different
```

	satu	dua
0	0	1
1	2	3
2	4	5

	tiga	empat
0	6	7
1	8	9
2	10	11

Concatenation: By Rows or By Columns

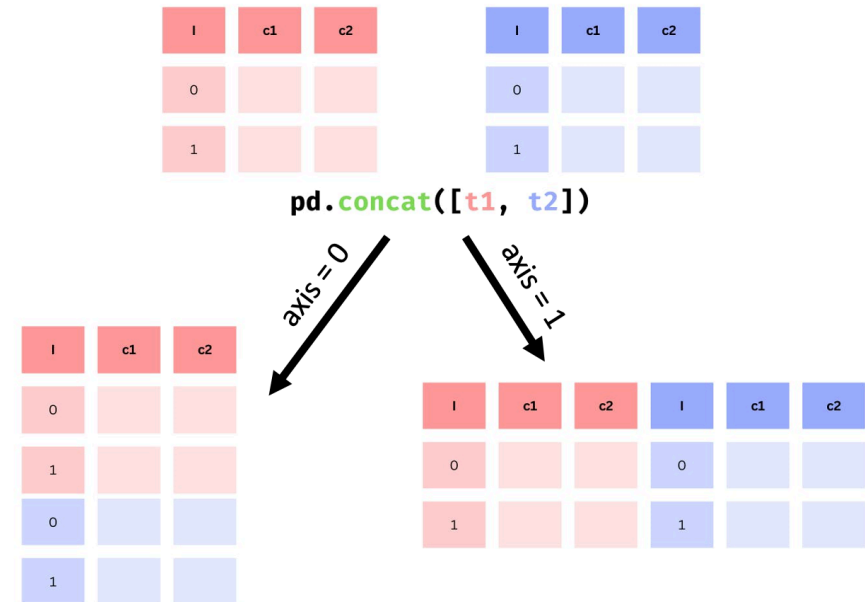
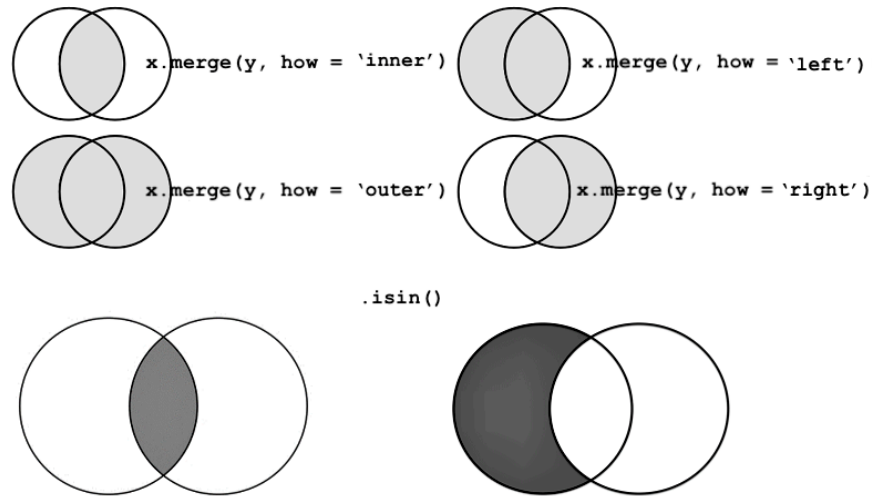
Let's take a look at the results of `.concat()` using `axis = 0` vs `axis = 1`:

```
1 df_row = pd.concat([df1, df2], axis = 0)
2 df_col = pd.concat([df1, df3], axis = 1)
3 print(df_row)
4 print(df_col)
```

	satu	dua
0	0	1
1	2	3
2	4	5
0	6	7
1	8	9
2	10	11

	satu	dua	tiga	empat
0	0	1	6	7
1	2	3	8	9
2	4	5	10	11

Summary



- **Joins/Merges:** Match key variables and keep columns of *both* inputs.
- **Row Filtering:** Match key variables and keep columns of *the first* input.
- **Concatenation:** Bind rows or columns from multiple tables.